RaDaR

Data Completeness Dashboard

Technical Documentation

Project	RaDaR Data Completeness Dashboard
Organisation	UK Kidney Association (UKKA)
Registry	Rare Disease Registry (RaDaR)
Author	Bidhan Pant
Version	1.0
Date	March 2026
Status	Prototype — demographics complete, cohorts in progress
Tech stack	Python, Pandas, Dash, PostgreSQL, SSH

1. Overview

The RaDaR Data Completeness Dashboard is an interactive web application that provides the UK Kidney Association with a real-time view of data quality across the Rare Disease Registry. It enables clinicians and data managers to quickly identify variables with poor completeness so that targeted data collection efforts can be prioritised.

The dashboard connects securely to the RaDaR PostgreSQL database via an SSH tunnel, calculates completeness for each variable, and presents results through a colour-coded accordion interface built with Python Dash. All 33 active disease cohorts are represented, with patient demographics fully implemented in the first release.

2. System Architecture

The system is structured in four layers, each with a single responsibility:

Layer	Responsibility
Data	PostgreSQL database on db.radar.nhs.uk accessed via SSH tunnel
Config	Variable definitions, connection settings, cohort metadata
Analytics	pandas scripts that query the DB and write completeness.json
Dashboard	Python Dash app that reads completeness.json and renders the UI

Data flows in one direction only: PostgreSQL -> SSH tunnel -> pandas -> completeness.json -> Dash dashboard. The dashboard never connects directly to the database at runtime — it reads from the pre-generated JSON file. This keeps the dashboard fast and decoupled from database availability.

3. Project Structure

The project follows a strict one-file-one-responsibility pattern. Every file has a clearly defined purpose and no file imports from a layer above it.

3.1 Root

File	Purpose
`app.py`	Entry point — creates and starts the Dash app on port 8050
`requirements.txt`	Python package dependencies with pinned versions
`.gitignore`	Excludes .env, output/, logs/, and virtual environment
`README.md`	Setup and usage instructions for new developers

3.2 config/

File	Purpose
`config/settings.py`	SSH tunnel class (pure paramiko) and psycopg2 connection factory
`config/demographics.py`	DEMOGRAPHICS_VARIABLES list and DEFAULT_EMAILS exclusion list
`config/cohorts.py`	33 cohort definitions (to be added for future cohort analytics)

3.3 analytics/

File	Purpose
`analytics/utils.py`	Shared functions: is_missing(), calc_pct_missing(), build_result(), logger setup

`analytics/demographics_completeness.py`	Main analytics script — connects, queries, calculates, writes completeness.json
`analytics/cohort_completeness.py`	Future — cohort-specific variable completeness (not yet implemented)

3.4 dashboard/

File	Purpose
`dashboard/__init__.py`	create_app() factory — initialises Dash, sets layout, registers callbacks
`dashboard/layout.py`	Page layout — assembles header, description, cards, legend, accordion
`dashboard/components/header.py`	Page header with RaDaR branding, live patient count, nav buttons
`dashboard/components/legend.py`	Colour key bar using QUOD thresholds
`dashboard/components/summary_cards.py`	4 KPI cards: total patients, complete, partial, need attention
`dashboard/components/table.py`	Colour-coded DataTable with QUOD thresholds
`dashboard/components/accordion.py`	Demographics + 33 cohort accordions with coming-soon placeholders
`dashboard/callbacks/data_callbacks.py`	Loads completeness.json into dcc.Store on page load and refresh
`dashboard/callbacks/render_callbacks.py`	Renders cards and accordion from store; handles expand/collapse
`dashboard/assets/style.css`	All CSS — header gradient, cards, accordion, legend, table

3.5 Other

File	Purpose
`output/completeness.json`	Generated by analytics script — never edit manually
`logs/radar.log`	Application and analytics logs with timestamps
`tests/test_analytics.py`	16 unit tests for calculation functions — no DB required
`tests/test_connection.py`	Integration tests for SSH tunnel and DB connection

4. Setup Guide

4.1 Prerequisites

- Windows 10/11 laptop
- Python 3.10+ installed
- Java 21 LTS (Temurin) — only needed if using PySpark in future
- SSH key at C:\Users\bidhan.pant\.ssh\id_rsa
- VPN/network access to db.radar.nhs.uk

4.2 Environment Variables

Set these once in Windows using Command Prompt. Close and reopen the terminal after running:

```
setx RADAR_DB_HOST "localhost"
setx RADAR_DB_PORT "5432"
setx RADAR_DB_NAME "radar"
setx RADAR_DB_USER "radar_ro"
setx RADAR_DB_PASS "your_password"
setx RADAR_SSH_HOST "db.radar.nhs.uk"
setx RADAR_SSH_PORT "22"
setx RADAR_SSH_USER "bidhanp"
setx RADAR_SSH_KEY "C:\Users\bidhan.pant\.ssh\id_rsa"
```

4.3 Install Dependencies

Create a virtual environment and install packages:

```
python -m venv radar_env
radar_env\Scripts\activate
pip install -r requirements.txt
```

The requirements.txt pins the following critical versions to avoid SSH tunnel breakage:

- paramiko==2.12.0 — newer versions break sshtunnel compatibility
- sshtunnel==0.4.0 — must be this version
- dash and dash-bootstrap-components — latest stable

5. Daily Usage

5.1 Running the Dashboard

Two commands are all that is needed:

```
python -m analytics.demographics_completeness
```

This connects to the database, calculates completeness for all demographics variables, and writes output/completeness.json. Takes approximately 30-60 seconds.

```
python app.py
```

This starts the Dash server. Open a browser and go to http://127.0.0.1:8050

5.2 Refreshing Data

While the dashboard is running, click the Refresh data button in the top navigation bar to reload completeness.json without restarting the server. To get genuinely fresh data from the database, run the analytics script again first.

6. Data Rules and Business Logic

6.1 Patient Population

Rule	Implementation
Source type	Only records where source_type = 'RADAR' are included
Test patients	Excluded where patients.test = TRUE
Control patients	Excluded where patients.control = TRUE
Join	patient_demographics joins to patients via patient_id = patients.id

6.2 Completeness Calculation

A value is considered missing if it is NULL or an empty/whitespace string. The is_missing() function in analytics/utils.py handles both cases:

```
missing = series.isna() | (series.astype(str).str.strip() == "")
```

6.3 Special Variables

Variable	Special Logic
EMAIL_ADDRESS	Missing if NULL or in the list of 47 known placeholder emails (e.g. radar@radar.org)
DIAGNOSIS	Missing if patient has no record in patient_diagnoses table (LEFT ANTI JOIN)
NHS_NUMBER	Missing if patient has no record in patient_numbers table (LEFT ANTI JOIN)
DATE_OF_DEATH	Informational only — shows count of deceased patients, not treated as missing
CAUSE_OF_DEATH	Calculated among deceased patients only (those with a non-null date_of_death)

6.4 Colour Thresholds (QUOD Standard)

Colour	Threshold
Green	0 to 19.9% missing
Yellow-green	20 to 39.9% missing
Yellow	40 to 59.9% missing
Orange	60 to 79.9% missing
Red	80 to 100% missing

7. Adding a New Cohort

When you are ready to add completeness data for a cohort group, follow these steps:

- Step 1 — Add the cohort variables to config/cohorts.py (create this file following the same pattern as config/demographics.py)
- Step 2 — Create analytics/cohort_completeness.py following the same pattern as demographics_completeness.py
- Step 3 — Update analytics/demographics_completeness.py to also run the cohort script and append results to completeness.json
- Step 4 — The dashboard accordion will automatically pick up the new section — no dashboard code changes needed

The JSON output format for a cohort section follows this structure:

```
{ "section": "B", "title": "Alport Syndrome", "variables": [
    { "id": "B.1", "name": "DIAGNOSIS_DATE", "pct_missing": 2.1,
      "missing": 87, "total": 4150, "required": true, "desc": "..." }
  ] }
```

8. Troubleshooting

Error	Fix
paramiko.DSSKey AttributeError	Run: pip install paramiko==2.12.0 --force-reinstall
sshtunnel error on init	Run: pip install sshtunnel==0.4.0 --force-reinstall
Environment variables show None	Close and reopen VS Code after running setx commands
Dashboard shows No data found	Run python -m analytics.demographics_completeness first
Duplicate callback ID error	Check header.py — refresh-btn must only appear once

DeprecationWarning in output	Already suppressed — add warnings.filterwarnings at top of script
Port 5433 already in use	Kill existing tunnel: netstat -ano | findstr :5433, then taskkill /PID <pid>

9. Known Limitations and Future Work

9.1 Current limitations

- Only patient demographics is fully implemented — 33 cohort variables are placeholders
- Dashboard reads from a static JSON file — data is only as fresh as the last analytics run
- No user authentication — dashboard is open to anyone on the local network
- No scheduled refresh — analytics script must be run manually

9.2 Planned future work

- Add variable-level completeness for all 33 cohorts
- Schedule analytics script to run nightly and refresh data automatically
- Add trend view — show completeness over time
- Add centre-level breakdown — completeness per recruiting centre
- Deploy to a shared server so the whole team can access the dashboard

10. Package Dependencies

Package	Version / Notes
pandas	Latest stable — data manipulation and SQL reading
psycopg2-binary	Latest stable — PostgreSQL connection
paramiko	2.12.0 PINNED — SSH key handling
sshtunnel	0.4.0 PINNED — SSH tunnel creation
dash	Latest stable — web dashboard framework
dash-bootstrap-components	Latest stable — Bootstrap layout components
python-dotenv	Latest stable — optional .env file support